université
PARIS-SACLAY

RAPPORT DE TP

Systèmes multitâches

Auteur :

Gaëtan BOUCHY
Moustafa KASSIR

Enseignant :

M. THOMAS

Nous attestons que ce travail est original, qu'il est le fruit d'un travail commun en binôme et qu'il a été rédigé de manière autonome.

À l'INSNT, le 22/11/2022

université
PARIS-SACLAY

Sommaire

Sommaire 2
1 - Préambule 3
1.1 - 1.8 Identifier le code et compléter le diagramme 3
2. Premier Exercice (Exercice Principal) 4
2.1 Conception détaillée en utilisant une approche dirigée par les événements 5
2.1.2 Une approche dirigée par les événements est-elle une approche synchrone ou asynchrone ? 5
2.1.3 À quelle stratégie de sûreté la méthode messageCheck répond-t-elle ? 5
2.1.4 À ce stade de l'implémentation squelette proposée, combien y a-t-il de processus (process) et de fil d'exécution (thread) POSIX dans ce programme ? 6
2.1.5 Complétez cette architecture logicielle détaillée par l'analyse des tâches à mettre en œuvre, des échanges de données et des synchronisations entre les tâches en satisfaisant les choix de conception précédents 7

1 - Préambule

1.1 - 1.8 Identifier le code et compléter le diagramme

1. Identifiez dans le programme le nom du thread : **thread_1**
2. Identifiez dans le programme le nom du sémaphore : **semaphore**
3. Identifiez dans le programme le nom du mutex : **mutex**
4. Identifiez la création du thread : **pthread_create(&thread_1, NULL, produce, NULL);**
5. Identifiez dans le programme le point d'entrée du thread : **void *produce(void *params)**
6. Identifiez l'attente de la fin du thread pour terminer le processus : **pthread_join(thread_1, NULL);**
7. Complétez le diagramme ci-dessous :

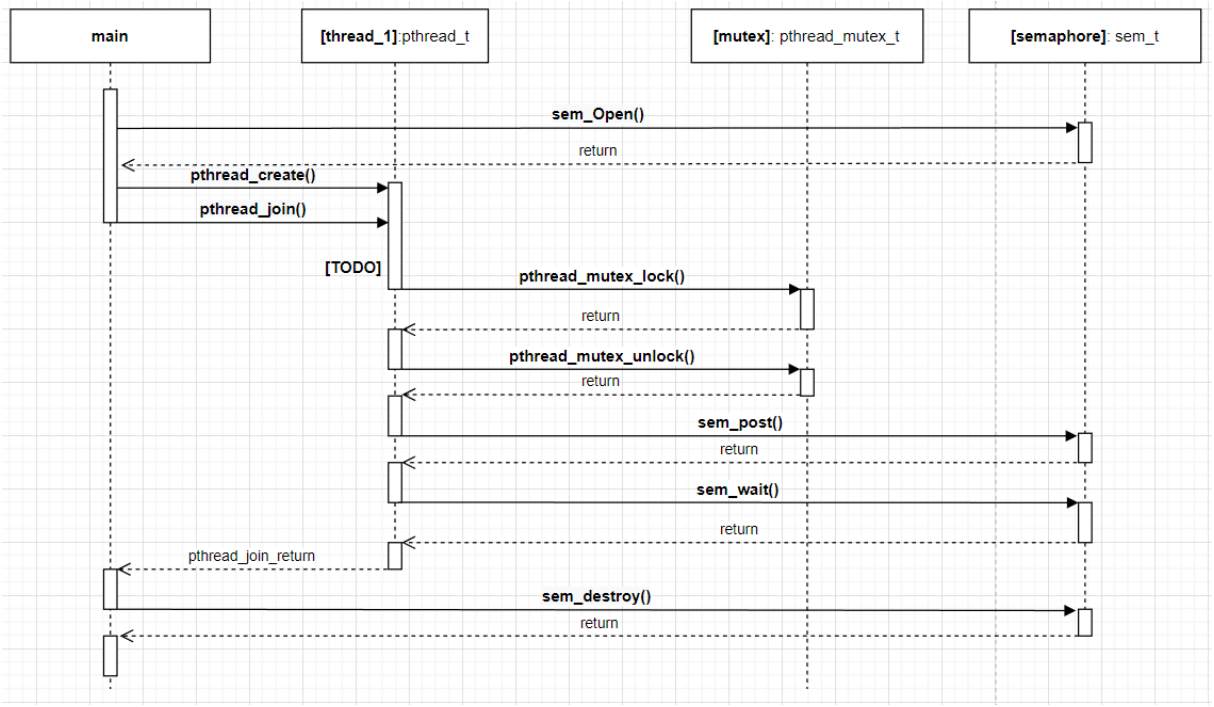


Figure 1 : Diagramme du fichier preambule.c

8. En partant de l'hypothèse qu'il n'existe pas d'exigences temporelles sur ce programme, sa conception est-elle complète ?

Afin de réaliser ce diagramme, nous avons suivi l'ordre chronologique des évènements du code car le langage C est un langage séquentiel. Cependant, la conception de ce programme n'est pas complète car le **pthread_exit(NULL)** n'est pas pris en compte dans le diagramme et il n'y a aucune gestion des erreurs. Ces méthodes renvoient donc des codes d'erreur ce qui n'est pas acceptable.

université
PARIS-SACLAY

2. Premier Exercice (Exercice Principal)

1. Complétez cette architecture logicielle en allouant les exigences de la spécification précédente sur les composants de **MultitaskingAccumulator**.

Nous avons donc 5 exigences que nous devons relier avec notre architecture logicielle :

- **Exigence 1** : **MultitaskingAccumulator** doit acquérir quatre entrées et produire en sortie le cumul des entrées dès qu'une entrée est acquise. Le cumul correspond à la production d'un tableau de 256 entiers dont le ieme élément de ce tableau est la somme des iemes éléments des tableaux d'entrée.

Nous voyons que cette exigence fait part de l'acquisition des entrées ce qui relève du rôle **d'AcquisitionManager** (**AcquisitionManager** acquiert les entrées depuis l'interface **Isensor**), mais également de la gestion de la sortie ce qui est le rôle de Display Manager (**DisplayManager** pilote la sortie) et enfin de la somme de chaque élément des tableaux de 256 entiers réalisée par **MessageAdder** (**MessageAdder** somme ces entrées). Ainsi, pour la première exigence, nous devons la relier avec **AcquisitionManager**, **DisplayManager** et **MessageAdder**.

- **Exigence 2** : **MultitaskingAccumulator** doit acquérir toutes les données d'entrées.

L'acquisition des données d'entrées n'est réalisée que par **AcquisitionManager.** Cette exigence concernant l'acquisition de **toutes** les données ne concerne donc que cette partie.

- Exigence 3 : **MultitaskingAccumulator** doit garantir que les données d'entrées sont correctement formées avant de les sommer.

Là encore, cette exigence qui traite des données d'entrées relève uniquement d'**AcquisitionManager**.

- Exigence 4 : **MultitaskingAccumulator** doit sommer et produire une sortie au plus vite, c'est-à-dire dès qu'une entrée est présente sans attendre une nouvelle valeur sur chacune des entrées.

Cette exigence traite de la rapidité à sommer ainsi que de la manière dont nous devons le faire. Nous savons que **MessageAdder** a pour but de sommer les entrées donc cette exigence le concerne uniquement.

- **Exigence 5** : Le logiciel doit produire sur la sortie diagnostic pour chaque opération de cumul, combien d'entrées ont été acquises, combien ont été sommées et combien restent à sommer.

Cette exigence traite de la sortie de notre programme donc de **DisplayManager,** mais pour l'obtention de ces informations, nous aurons également besoin de **MessageAdder afin** qu'il communique avec **DisplayManager**.

Ainsi, voici le diagramme final avec les différentes exigences reliées à notre architecture que nous obtenons :

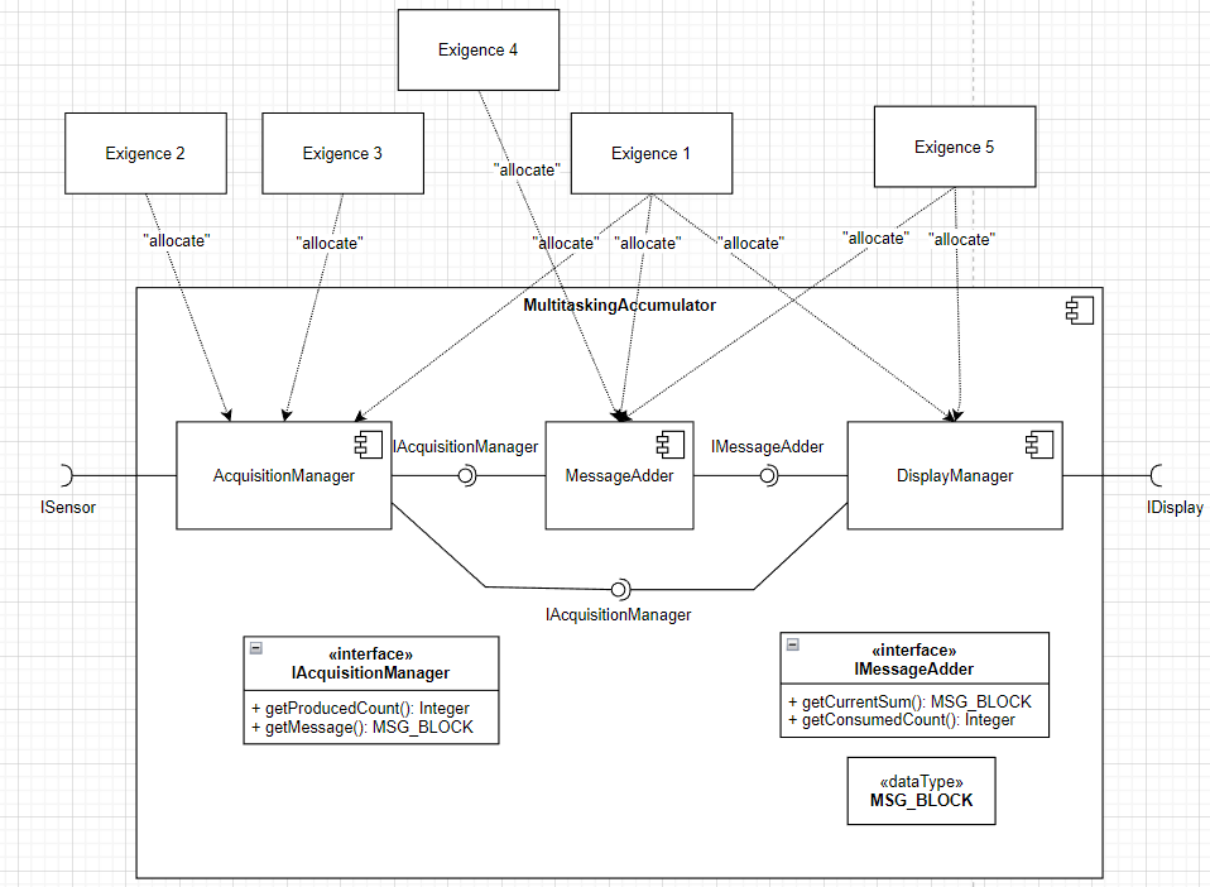


Figure 2 : Architecture logicielle avec allocation des exigences

2.1 Conception détaillée en utilisant une approche dirigée par les événements

2.1.2 Une approche dirigée par les événements est-elle une approche synchrone ou asynchrone ?

Une approche dirigée par les évènements est une approche asynchrone. Dans ce cas-là, nous une meilleure réactivité de notre système (vis-à-vis des interruptions) par rapport à une approche synchrone (approche dirigée par le temps). Cependant le déterminisme n'est pas assuré par la construction et le temps d'exécution de bout en bout est imprévisible et dépend des performances du matériel pour une approche asynchrone.

2.1.3 À quelle stratégie de sûreté la méthode messageCheck répond-t-elle ?

Le programme **messageCheck()**, assure la validité du message transmis. Pour cela, il réalise un OU bit-à-bit exclusif avec tous les éléments du tableau de 256 entiers puis compare ce résultat avec celui du Checksum. Checksum qui a été préalablement calculé dans l'interface **Isensor** via le programme **getInput()** simulant la création des entrées. Si ces deux résultats correspondent, cela signifie que le message n'a pas été corrompu.

Ce programme ferait donc parti de la stratégie de sûreté concernant la « **détection des erreurs** » (Failure Detection).

2.1.4 À ce stade de l'implémentation squelette proposée, combien y a-t-il de processus (process) et de fil d'exécution (thread) POSIX dans ce programme ?

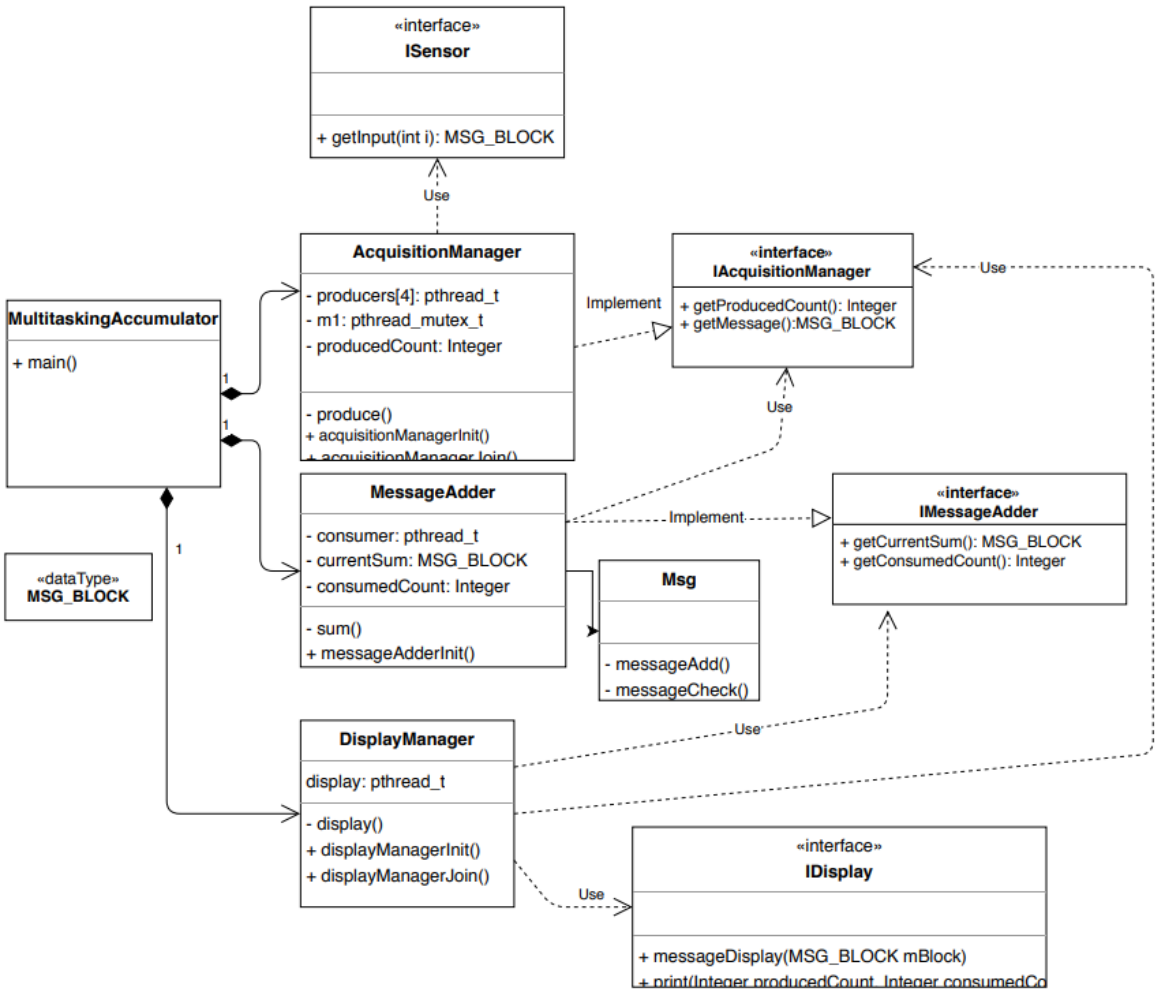


Figure 3 : Architecture à ce niveau d'implémentation

À ce stade-là, il n'y a qu'un seul processus et il n'y en aura toujours un malgré les modifications que nous ferons. Il peut y avoir plusieurs instances d'un même programme, et chaque instance de ce programme en cours d'exécution est appelé un processus. Chaque processus dispose d'un espace d'adressage mémoire distinct, ce qui signifie qu'un processus fonctionne indépendamment et est isolé des autres processus.

Concernant le nombre de threads POSIX dans ce programme, il y en a actuellement 6. Comme nous pouvons le voir, quatre d'entre eux proviennent d'**AcquisitionManager** (**producers[4] : pthread_t**), un de **MessageAdder** (**consumer : pthread_t**) et enfin le dernier se trouve dans **DisplayManager** (**display : pthread_t**). Ce nombre de 6 threads a été défini pour cette architecture incomplète, il se peut donc qu'il soit amené à changer avec de futures modifications.

2.1.5 Complétez cette architecture logicielle détaillée par l'analyse des tâches à mettre en œuvre, des échanges de données et des synchronisations entre les tâches en satisfaisant les choix de conception précédents

Nous avons donc dû compléter l'architecture logicielle précédente que nous avons vu dans la partie (2.1.4 À ce stade de l'implémentation squelette proposée, combien y a-t-il de processus (process) et de fil d'exécution (thread) POSIX dans ce programme ?)

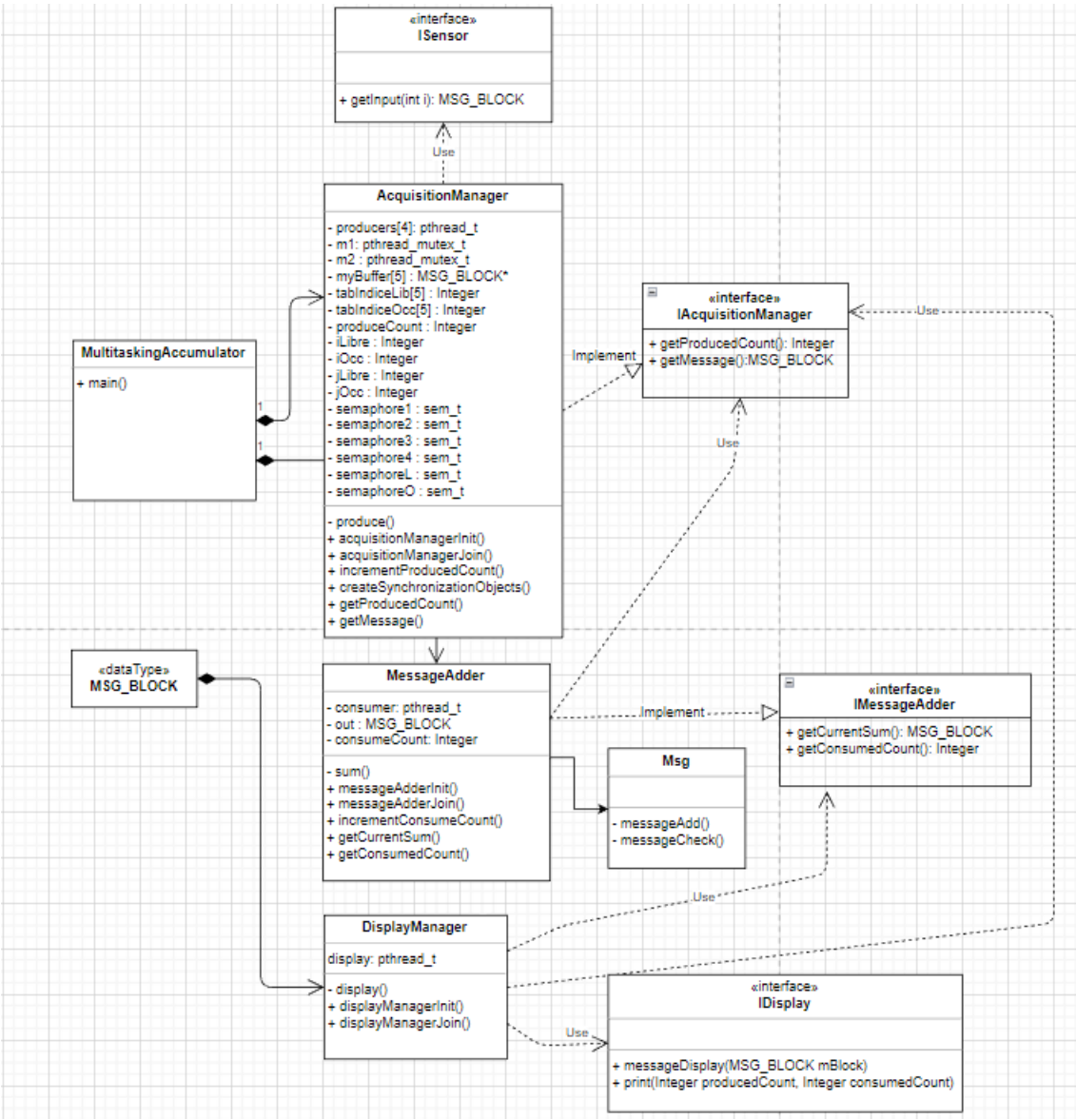


Figure 4 : Architecture logicielle détaillée par l'analyse des tâches à mettre en œuvre, des échanges de données et des synchronisations entre les tâches en satisfaisant les choix de conception précédents

Dans cette nouvelle architecture, nous avons modifié la partie concernant **messageAdder** où nous avons rajouté les fonctions manquantes et la partie **AcquisitionManager** :

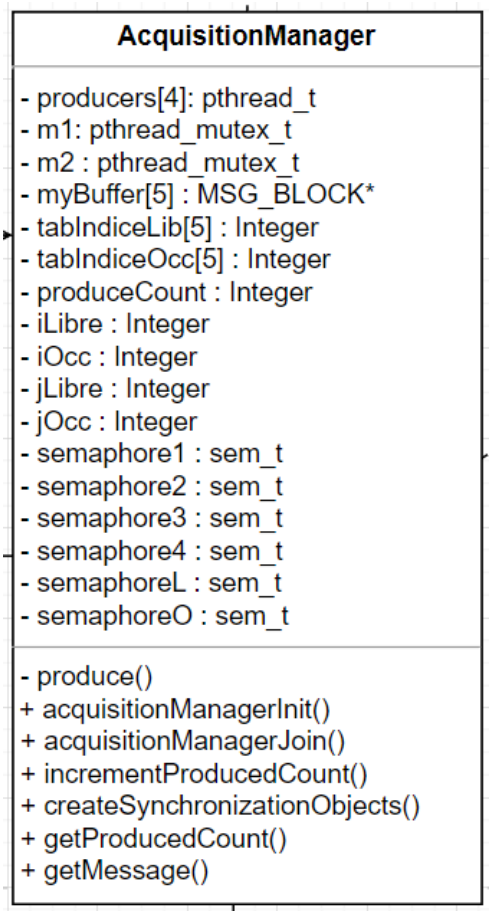


Figure 5 : Architecture logicielle détaillée d'AcquisitionManager

Dans cette partie, nous avons tout d'abord rajouté un tableau de pointeur de MSG_BLOCK. Ce tableau de pointeur va nous être utile car nous y allons y stocker les adresses des entrées. Nous allons donc y écrire dedans si la condition du « checksum » est vérifiée et si le buffer n'est pas complet (toutes les cases sont remplies).

Nous avons également rajouté deux tableaux pour les indices libres et les indices occupés ainsi que quatre variables (iLibre, iOcc, jLibre, jOcc) afin de réaliser le multi-write c'est-à-dire que plusieurs threads peuvent écrire en même temps dans le buffer mais dans des cases libres différentes. De plus, cela va également nous servir pour faire la lecture en mode single-read c'est-à-dire qu'un seul thread uniquement va pouvoir lire notre buffer (dans l'une des cases où nous ne sommes pas en train d'écrire).

Ainsi pour faire le multiple-write, single-read, nous avons eu besoin de sémaphores, 6 en particulier, deux spécifiquement dédiés pour le multi-write, deux spécifiquement dédiés pour le single-read et deux partagés. Nous aurons également deux mutex pour protéger l'incrémentation de **produceCount** et le **getProducedCount()**.

2.1.6 Implémentez votre conception (Implémentation + Exécution) et montrez un résultat d'exécution

Afin de réaliser la conception de notre code en fonction de l'architecture logicielle détaillée. Nous avons dû compléter les fichiers **acquisitionManagerPosix.c**, **messageAdder.c** et **displayManager.c** de notre programme.

Pour le programme **acquisitionManagerPosix.c** qui réalise l'acquisition des entrées, nous avons ainsi dû compléter les fonctions suivantes :

```
unsigned int acquisitionManagerInit(void)
{
        unsigned int i;
        printf("[acquisitionManager]Synchronization initialization in progress...\n");
        fflush( stdout );
        if (createSynchronizationObjects() == ERROR_INIT)
                return ERROR_INIT;

        printf("[acquisitionManager]Synchronization initialization done.\n");

        for (i = 0; i < PRODUCER_COUNT; i++)
        {
                pthread_create(&producers[i], NULL, produce, NULL);
        }

        return ERROR_SUCCESS;
}
```

Figure 6 : Fonction acquisitionManagerInit()

Cette fonction permet la création des quatre threads de notre tableau pthread_t producers[4].

```
void acquisitionManagerJoin(void)
{
        unsigned int i;
        for (i = 0; i < PRODUCER_COUNT; i++)
        {
                pthread_join(producers[i], NULL);
        }

        sem_destroy(semaphore1);
        sem_destroy(semaphore2);
        sem_destroy(semaphore3);
        sem_destroy(semaphore4);
        sem_destroy(semaphoreL);
        sem_destroy(semaphore0);
        printf("[acquisitionManager]Semaphore cleaned\n");
}
```

Figure 7 : Fonction acquisitionManagerJoin()

Cette fonction réalise **pthread_join()** qui fournit un mécanisme permettant à une application d'attendre la fin d'un thread pour les quatre threads de notre tableau ainsi que la destruction de nos 6 sémaphores

```c
static unsigned int createSynchronizationObjects(void)
{
    sem_unlink(SEM_NAME1);
    sem_unlink(SEM_NAME2);
    sem_unlink(SEM_NAME3);
    sem_unlink(SEM_NAME4);
    sem_unlink(SEM_NAME5);
    sem_unlink(SEM_NAME6);
    semaphore1 = sem_open(SEM_NAME1, O_CREAT, 0644, SEMAPHORE_INITIAL_VALUE);
    semaphore2 = sem_open(SEM_NAME2, O_CREAT, 0644, SEMAPHORE_INITIAL_VALUE);
    semaphore3 = sem_open(SEM_NAME3, O_CREAT, 0644, SEMAPHORE_INITIAL_VALUE);
    semaphore4 = sem_open(SEM_NAME4, O_CREAT, 0644, SEMAPHORE_INITIAL_VALUE);
    semaphoreL = sem_open(SEM_NAME5, O_CREAT, 0644, BUFFER_SIZE);
    semaphoreO = sem_open(SEM_NAME6, O_CREAT, 0644, SEMAPHORE_INITIAL_VALUE);

    if ((semaphore1 == SEM_FAILED) || (semaphore2 == SEM_FAILED) || (semaphore3 == SEM_FAILED) ||
        (semaphore4 == SEM_FAILED) || (semaphoreL == SEM_FAILED) || (semaphoreO == SEM_FAILED))
    {
        perror("[sem_open]");
        return ERROR_INIT;
    }

    for(int i=0; i<BUFFER_SIZE; i++)
    {
        tabIndiceOcc[i]=i;
        tabIndiceLib[i]=i;
    }

    return 0;
}
```

Figure 8 : Fonction createSynchronizationObjects

Cette fonction va simplement initialiser nos sémaphores ainsi que remplir les tableaux des indices occupés et des indices libres.

```c
void *produce(void* params)
{
        int iBuffer;


        D(printf("[acquisitionManager]Producer created with id %d\n", gettid()));
        unsigned int i = 0;
        MSG_BLOCK temp;

        while (i < PRODUCER_LOOP_LIMIT)
        {
                sem_wait(semaphoreL); // Baisse de valeur le nombre de case libre
                i++;
                sleep(PRODUCER_SLEEP_TIME+(rand() % 5));
                getInput(i,&temp);

                if(messageCheck(&temp)==1)
                {
                        // CODE MULTI-WRITE
                        sem_post(semaphore1);
                                iBuffer = tabIndiceLib[iLibre];
                                iLibre = ((iLibre + 1)%BUFFER_SIZE);
                        sem_wait(semaphore1);

                        myBuffer[iBuffer] = &temp; //Je place dans mon buffer mon tempon
                        incrementProducedCount(); // J'incrémente le nombre d'éléments produit
                        printf("MESSAGE PRODUIT : %d\n", getProducedCount());

                        sem_post(semaphore2);
                                tabIndiceOcc[iOcc]= iBuffer;
                                iOcc = ((iOcc + 1)%BUFFER_SIZE);
                        sem_wait(semaphore2);
                }
                sem_post(semaphoreO); // Augmente de valeur le nombre de case occupée
        }
        D(printf("[acquisitionManager] %d termination\n", gettid()));

        pthread_exit(NULL);
}
```

Figure 9 : Fonction produce()

La fonction **produce()** va tout d'abord appeler **getInput()** afin de simuler la production d'une entrée. Puis la condition du **if()** va vérifier que le checksum est validé (le message n'est pas corrompu). Si cette condition est vérifiée, nous allons écrire dans notre buffer au bon endroit grâce à la gestion des indices puis nous allons incrémenter **producedCount** via la fonction **incrementProducedCount()**.

Nous diminuons le nombre de case libre (**sem_wait(semaphoreL)**) et augmentons le nombre de case occupée (**sem_post(semaphoreO)**).

```c
static void incrementProducedCount(void)
{
        pthread_mutex_lock(&m1);
        produceCount++;
        pthread_mutex_unlock(&m1);
}

unsigned int getProducedCount(void)
{
        unsigned int p = 0;
        p = produceCount;
        return p;
}
```

Figure 10 : Fonction incrementProduceCount() et getProducedCount()

Comme son nom l'indique, la fonction **incrementProducedCount()** incrémente le nombre de messages produits. La variable est protégée par un mutex pour que deux threads n'incrémentent pas cette variable de manière simultanée.

La fonction **getProducedCount()** va simplement renvoyer la valeur de **produceCount** donc le nombre de messages crées.

```c
MSG_BLOCK getMessage(void)
{
        int jBuffer;
        MSG_BLOCK* temp;

        sem_wait(semaphoreO); // Diminue le nombre de case occupée

        sem_post(semaphore3);
                jBuffer = tabIndiceOcc[jOcc];
                jOcc = ((jOcc + 1)%BUFFER_SIZE);
        sem_wait(semaphore3);

        temp = myBuffer[jBuffer];
        consommeCount++;
        printf("MESSAGE CONSOMME : %d\n",consommeCount);

        sem_post(semaphore4);
                tabIndiceLib[jLibre] = jBuffer;
                jLibre = ((jLibre + 1)%BUFFER_SIZE);
        sem_wait(semaphore4);

        sem_post(semaphoreL); // augmente de valeur le nombre de case libre

        return *temp;
}
```

Figure 11 : Fonction getMessage()

La fonction **getMessage()** va récupérer un élément du buffer grâce à la gestion des indices puis va renvoyer sa valeur avec le mot clé « ***** » car c'est un pointeur. Nous diminuons le nombre de case occupée (**sem_wait(semaphoreO)**) et augmentons le nombre de case libre (**sem_post(semaphoreL)**).

Pour le programme **messageAdder.c** qui somme les entrées, nous avons ainsi dû compléter les fonctions suivantes :

```
void messageAdderInit(void){
        out.checksum = 0;
        for (size_t i = 0; i < DATA_SIZE; i++)
        {
                out.mData[i] = 0;
        }
        pthread_create(&consumer, NULL, sum, NULL);
}

void messageAdderJoin(void)
{
        pthread_join(consumer, NULL);
}
```

Figure 12 : Fonction messageAdderInit() et messageAdderJoin()

Ces deux fonctions servent pour la création du thread (pthread_t consumer) puis **pthread_join()** qui fournit un mécanisme permettant à une application d'attendre la fin d'un thread.

```c
static void *sum( void *parameters )
{
        MSG_BLOCK msg;
        D(printf("[messageAdder]Thread created for sum with id %d\n", gettid()));
        unsigned int i = 0;
        while(i<ADDER_LOOP_LIMIT)
        {
                i++;
                sleep(ADDER_SLEEP_TIME);
                msg = getMessage();
                incrementConsumeCount();
                messageAdd(&out, &msg);
        }
        printf("[messageAdder] %d termination\n", gettid());
        pthread_exit(NULL);
}
```

Figure 13 : Fonction sum()

La fonction **sum()** va recevoir un message via la fonction **getmessage()**, puis cela va incrémenter **consumeCount** via la fonction **incrementConsumeCount()**. Enfin, nous allons ajouter nos entrées grâce à **messageAdd()**.

```c
MSG_BLOCK getCurrentSum()
{
        return out;
}

unsigned int getConsumedCount()
{
        return consumeCount;
}

static void incrementConsumeCount(void)
{
        consumeCount++;
}
```

Figure 14 : Fonction getCurrentSum(), getConsumedCount() et incrementConsumeCount()

Comme son nom l'indique, **getCurrentSum()** permet de retourner un MSG_BLOCK qui contient la somme actuelle. **getConsumedCount()** retourne le nombre de messages consommés et **incrementConsumeCount()** incrémente le nombre de messages consommés.

Pour le programme **displayManager.c** qui pilote la sortie, nous avons ainsi dû compléter les fonctions suivantes :

```
void displayManagerInit(void)
{
        pthread_create(&displayThread, NULL, display, NULL);
}

void displayManagerJoin(void)
{
        pthread_join(displayThread, NULL);
}
```

Figure 15 : Fonction displayManagerInit() et displayManagerJoin()

Ces deux fonctions servent pour la création du thread (pthread_t display) puis **pthread_join()** qui fournit un mécanisme permettant à une application d'attendre la fin d'un thread.

```
static void *display( void *parameters )
{
        MSG_BLOCK mess;
        D(printf("[displayManager]Thread created for display with id %d\n", gettid()));
        unsigned int diffCount = 0;
        while(diffCount < DISPLAY_LOOP_LIMIT)
        {
                sleep(DISPLAY_SLEEP_TIME);
                mess = getCurrentSum();
                messageDisplay(&mess);
                print(getProducedCount(), getConsumedCount());

                if(getProducedCount()-getConsumedCount()==0)
                        diffCount = DISPLAY_LOOP_LIMIT;
        }
        printf("[displayManager] %d termination\n", gettid());
        pthread_exit(NULL);
}
```

Figure 16 : Fonction display()

La fonction display() va récupérer la somme, nous allons afficher le message puis le nombre

```
[multitaskingAccumulator]Software initialization in progress...
[acquisitionManager]Synchronization initialization in progress...
[acquisitionManager]Synchronization initialization done.
[multitaskingAccumulator]Task initialization done.
[multitaskingAccumulator]Scheduling in progress...
[OK      ] Checksum validated
[OK      ] Checksum validated
[OK      ] Checksum validated
[OK      ] Checksum validated
Message
[displayManager]Produced messages: 2, Consumed messages: 1, Messages left: 1
[OK      ] Checksum validated
[OK      ] Checksum validated
[OK      ] Checksum validated
[OK      ] Checksum validated
[OK      ] Checksum validated
Message
[displayManager]Produced messages: 7, Consumed messages: 2, Messages left: 5
[OK      ] Checksum validated
[OK      ] Checksum validated
Message
[displayManager]Produced messages: 8, Consumed messages: 4, Messages left: 4
[OK      ] Checksum validated
Message
[displayManager]Produced messages: 8, Consumed messages: 6, Messages left: 2
[OK      ] Checksum validated
Message
[displayManager]Produced messages: 8, Consumed messages: 7, Messages left: 1
[messageAdder] 3942 termination
[OK      ] Checksum validated
Message
[displayManager]Produced messages: 8, Consumed messages: 8, Messages left: 0
[displayManager] 3943 termination
[acquisitionManager]Semaphore cleaned
[multitaskingAccumulator]Threads terminated
```

Figure 17 : Résultat de l'implémentation de notre conception avec un buffer de taille 5

Comme nous pouvons le voir, nous avons en effet produit 8 messages et également consommé 8. Notre buffer étant de taille 5, nous n'avons jamais réécrit sur une variable non lue car nous avons d'abord produit deux messages puis consommé l'un d'entre eux. Puis nous avons produit trois messages et consommé l'un d'entre eux. Puis nous en avons encore produit deux puis consommé deux. Cela est dû au sémaphore **semaphoreL** qui vaut la taille du buffer et qui va diminuer d'un à chaque fois que l'on écrit dans le buffer et augmenter d'un à chaque fois que l'on lit le buffer. Ainsi, si nous avons écrit dans toutes les cases du buffer, nous ne pourrons plus y écrire car le **semaphoreL** vaudra 0

Avec un buffer de taille 1, nous voyons que chaque message produit est ensuite consommé grâce au **semaphoreL** :

```
[multitaskingAccumulator]Software initialization in progress...
[acquisitionManager]Synchronization initialization in progress...
[acquisitionManager]Synchronization initialization done.
[multitaskingAccumulator]Task initialization done.
[multitaskingAccumulator]Scheduling in progress...
[OK      ] Checksum validated
Message
[displayManager]Produced messages: 0, Consumed messages: 0, Messages left: 0
[displayManager] 2591 termination
[OK      ] Checksum validated
MESSAGE PRODUIT : 1
MESSAGE CONSOMME : 1
[OK      ] Checksum validated
MESSAGE PRODUIT : 2
MESSAGE CONSOMME : 2
[OK      ] Checksum validated
MESSAGE PRODUIT : 3
MESSAGE CONSOMME : 3
[OK      ] Checksum validated
MESSAGE PRODUIT : 4
MESSAGE CONSOMME : 4
[OK      ] Checksum validated
MESSAGE PRODUIT : 5
MESSAGE CONSOMME : 5
[OK      ] Checksum validated
MESSAGE PRODUIT : 6
MESSAGE CONSOMME : 6
[OK      ] Checksum validated
MESSAGE PRODUIT : 7
MESSAGE CONSOMME : 7
[OK      ] Checksum validated
MESSAGE PRODUIT : 8
MESSAGE CONSOMME : 8
[messageAdder] 2590 termination
[acquisitionManager]Semaphore cleaned
[multitaskingAccumulator]Threads terminated
```

Figure 18 : Résultat de l'implémentation de notre solution avec un buffer de taille 1

2.1.7 Pourquoi certaines variables sont-elles considérées comme des variables C volatiles dans l'implémentation proposée ?

Certaines variables sont considérées comme des variables volatiles pour prévenir le compilateur de faire des optimisations vis-à-vis de cette variable. Autrement dit, de prévenir le compilateur que cette variable peut être changée par plusieurs threads.

2.1.8 Nous avons orienté l'architecture détaillée avec l'utilisation des tâches. Nous aurions pu utiliser des processus POSIX. Citez les caractéristiques intéressantes des processus pour une conception orientée « sûreté de fonctionnement » ? Citez également celles qui ne sont pas satisfaites ?

Les processus sont caractérisés par un espace mémoire unique non partagé avec les autres processus, c'est pour cela ce n'est pas nécessaire de protéger les ressources. Ceci permet aussi d'avoir un confinement des erreurs puisqu'on a un partitionnement spatial de la mémoire.

L'utilisation des processus est coûteuse en mémoire et en temps puisque chaque processus a son propre espace mémoire qui ne peut être utilisé par d'autres processus, ce qui rend la communication entre les processus plus coûteuse que la communication entre les threads.

Si on avait travaillé avec les processus, notre conception n'aurait pas besoin de sémaphores pour protéger la ressource principale qui est le buffer et la synchronisation entre les threads car chaque processus aurait une mémoire propre à lui.

2.1.9 Proposez une solution pour protéger de manière efficace entre les tâches et sans utilisez des apis POSIX, le compteur permettant de compter le nombre de messages produits ?

Nous proposons l'utilsation du mot clé _Atomic tel que nous aurons : **_Atomic volatile unsigned int produce_count.** Cela garantirait l'atomicité de l'accès pour la variable **produce_count**.

2.1.10 Implémentez cette solution dans la méthode incrementProducerCount et getProducerCount dans le fichier acquisitionManagerAtomic.c et montrez un résultat d'exécution

Ainsi, nous avons rajouté le mot clé _Atomic devant la déclaration de la variable **produceCount**

```
_Atomic int produceCount = 0;
```

Figure 19 : Déclaration atomique de produceCount

Notre fonction incrementProducedCount() n'aura donc plus de mutex car produceCount est « _Atomic ».

```
static void incrementProducedCount(void)
{
        produceCount++;
}
```

Figure 20 : Fonction incrementProducedCount()

```
cc -pthread -o multitaskingAccumulatorAtomic acquisitionManagerAtomic.o display.o displayManager.o messageAdder.o msg.o multitasking
Accumulator.o sensorManager.o
./multitaskingAccumulatorAtomic
[multitaskingAccumulator]Software initialization in progress...
[acquisitionManager]Synchronization initialization in progress...
[acquisitionManager]Synchronization initialization done.
[multitaskingAccumulator]Task initialization done.
[multitaskingAccumulator]Scheduling in progress...
[OK      ] Checksum validated
[OK      ] Checksum validated
[OK      ] Checksum validated
Message
[displayManager]Produced messages: 2, Consumed messages: 1, Messages left: 1
[OK      ] Checksum validated
[OK      ] Checksum validated
[OK      ] Checksum validated
[OK      ] Checksum validated
[OK      ] Checksum validated
Message
[displayManager]Produced messages: 6, Consumed messages: 3, Messages left: 3
[OK      ] Checksum validated
[OK      ] Checksum validated
Message
[displayManager]Produced messages: 7, Consumed messages: 4, Messages left: 3
[OK      ] Checksum validated
[OK      ] Checksum validated
Message
[displayManager]Produced messages: 8, Consumed messages: 6, Messages left: 2
[OK      ] Checksum validated
Message
[displayManager]Produced messages: 8, Consumed messages: 7, Messages left: 1
[messageAdder] 4442 termination
[OK      ] Checksum validated
Message
[displayManager]Produced messages: 8, Consumed messages: 8, Messages left: 0
[displayManager] 4443 termination
[acquisitionManager]Semaphore cleaned
[multitaskingAccumulator]Threads terminated
gaetan@gaetan-VirtualBox:~/Bureau/TP1$ make runatomic
```

Figure 21 : Résultat d'exécution avec produceCount qui est « _Atomic »

2.1.11 Proposez une autre solution pour que ces méthodes incrementProducerCount et getProducerCount en vous basant sur la méthode atomic_compare_exchange_weak ?

D'après la documentation officielle, cela compare de manière atomique le contenu de la mémoire pointée par obj avec le contenu de la mémoire pointée par expected, et si ceux-ci sont égaux au niveau du bit, remplace le premier par desired (effectue une opération de lecture-modification-écriture). Sinon, charge le contenu actuel de la mémoire pointée par obj dans *expected (effectue une opération de chargement). Le programme se déroule ainsi :

Si lock = expected avec expected = 1
    Alors lock = desired
    return true
Sinon
    expected = lock
    return false

2.1.14 Une approche dirigée par le temps est-elle une approche synchrone ou asynchrone ?

Une approche dirigée par le temps (donc par une horloge) est une approche synchrone. Dans ce cas-là, nous avons :

- Les contraintes temporelles sont données lors de l'étape de conception
- Les données communiquées sont horodatées lors de la conception
- Tous les traitements sont délimités entre deux instants
- Tous les transferts de données sont effectués à un point de synchronisation temporelle, c'est ce qu'on appelle le partitionnement temporel
- La charge du CPU pour une tâche est bien définie

Cependant, la latence pour un système synchrone est plus élevée que pour un système asynchrone et donc la gestion des interruptions est moins rapide.

3. Second Exercice

3.1 Quelles sont les ressources partagées qui synchronisent le repas des philosophes

Les ressources partagées sont les fourchettes car chaque philosophe a besoin de deux fourchettes pour manger. Il y a cinq philosophes et cinq fourchettes il est nécessaire d'avoir deux fourchettes pour manger. Ainsi, seulement deux philosophes pourront manger simultanément.

3.2 Mettez en évidence une configuration d'interblocage avec les 5 philosophes présents à chaque place de la table. Donnez le scénario qui permet d'atteindre cet interblocage

Pour interbloquer les cinq philosophes, il faut qu'il ait tous une fourchette dans leur main. Cela peut arriver car le philosophe prend d'abord sa propre fourchette à sa gauche puis celle à sa droite. Ainsi, si les cinq philosophes saisissent leur fourchette à leur gauche en même temps, ils se retrouveront tous avec une fourchette et la situation ne pourra évoluer (graphe de pétri : réseau mort / réseau bloqué).

3.3 Mettez en évidence une configuration d'interblocage avec les 5 philosophes présents à chaque place de la table. Donnez le scénario qui permet d'atteindre cet interblocage

Pour éviter l'interblocage, il suffit d'ajouter une fourchette supplémentaire. Ainsi avec six fourchettes au total et cinq philosophes, il y aura forcément un philosophe qui aura deux fourchettes et ainsi la situation ne sera plus bloquée.

Une deuxième solution serait que les philosophes récupèrent les deux fourchettes de manière simultanée.